

Assignment No. 05

- **TITLE** – Represent polynomials using both arrays of coefficients and exponents and arrays of structures to explore different data representations. Implement polynomial addition and multiplication operations to simulate real-world computations. Analyze how the choice of representation affects the efficiency and flexibility of polynomial operations in practical applications.

- **THEORY** –

Polynomials are fundamental mathematical expressions widely used in various computational domains including computer graphics, signal processing, and scientific computing. The representation of polynomials in computer programs presents an interesting trade-off between memory efficiency and computational performance. The array-based representation stores coefficients in an array where the index corresponds to the exponent, making it highly efficient for dense polynomials with sequential exponents. This approach offers $O(1)$ access time for any coefficient and simplifies operations like evaluation but suffers from memory inefficiency when dealing with sparse polynomials where many coefficients are zero.

The structure-based representation, using term objects containing coefficient-exponent pairs, provides superior memory efficiency for sparse polynomials and can handle non-sequential exponents effectively. However, this approach requires more complex algorithms for arithmetic operations and may need sorting to maintain canonical form. The choice between these representations depends on the specific application requirements: array representation excels in numerical computations and dense polynomial operations, while structure representation is preferable for symbolic mathematics, sparse polynomials, and memory-constrained environments. Both approaches demonstrate how data structure selection directly impacts algorithm efficiency and resource utilization in practical applications.

- **Solved Example** –

$$P(x) = 3x^2 + 2x + 1$$

$$Q(x) = 2x + 3$$

Step 1: Array Representation

$$P(x) = [1, 2, 3] \text{ // index 0: coefficient of } x^0, \text{ index 1: } x^1, \text{ index 2: } x^2$$

$$Q(x) = [3, 2, 0] \text{ // } 2x + 3 = 3 + 2x + 0x^2$$

Step 2: Polynomial Addition

$$\text{Result}[i] = P[i] + Q[i] \text{ for all } i$$

$$\text{Result}[0] = 1 + 3 = 4$$

$$\text{Result}[1] = 2 + 2 = 4$$

$$\text{Result}[2] = 3 + 0 = 3$$

$$\therefore P(x) + Q(x) = 3x^2 + 4x + 4$$

Step 3: Polynomial Multiplication

For each term in P, multiply with each term in Q:

$$3x^2 \times 2x = 6x^3$$

$$3x^2 \times 3 = 9x^2$$

$$2x \times 2x = 4x^2$$

$$2x \times 3 = 6x$$

$$1 \times 2x = 2x$$

$$1 \times 3 = 3$$

Combine like terms:

$$6x^3 + (9x^2 + 4x^2) + (6x + 2x) + 3 \\ = 6x^3 + 13x^2 + 8x + 3$$

Array multiplication:

$$\text{Result}[0] = 1 \times 3 = 3$$

$$\text{Result}[1] = (1 \times 2) + (2 \times 3) = 2 + 6 = 8$$

$$\text{Result}[2] = (2 \times 2) + (3 \times 3) = 4 + 9 = 13$$

$$\text{Result}[3] = 3 \times 2 = 6$$

$$\therefore P(x) \times Q(x) = 6x^3 + 13x^2 + 8x + 3$$

$$\text{Addition: } (3x^2 + 2x + 1) + (2x + 3) = 3x^2 + 4x + 4$$

$$\text{Multiplication: } (3x^2 + 2x + 1)(2x + 3) = 6x^3 + 13x^2 + 8x + 3$$

➤ ALGORITHM –

1. Array Representation Algorithm

Initialize Polynomial

STEP 1: Declare array poly[MAX_DEGREE+1]

STEP 2: Initialize all elements to 0

STEP 3: Read degree from user

STEP 4: For i = degree down to 0:

Read coefficient for poly[i]

Display Polynomial

STEP 1: For i = MAX_DEGREE down to 0:

IF poly[i] ≠ 0:

PRINT poly[i]

IF i > 0: PRINT "x^" + i

IF i ≠ 0: PRINT "+"

2. Structure Representation Algorithm

Initialize Polynomial

STEP 1: Define Term structure {coeff, exp}

STEP 2: Declare array of Term poly[MAX_TERMS]

STEP 3: For each term:

Read coefficient and exponent

STEP 4: Sort terms by exponent (descending)

3. Polynomial Addition Algorithm

Array Representation

STEP 1: Find $\text{max_degree} = \max(\text{deg1}, \text{deg2})$

STEP 2: Declare $\text{result}[\text{MAX_DEGREE}+1]$

STEP 3: For $i = 0$ to max_degree :

$\text{result}[i] = \text{poly1}[i] + \text{poly2}[i]$

Structure Representation

STEP 1: Initialize empty result list

STEP 2: While both polynomials have terms:

IF $\text{exp1} == \text{exp2}$:

ADD $\text{coeff1} + \text{coeff2}$ to result

MOVE to next terms in both

ELSE IF $\text{exp1} > \text{exp2}$:

ADD term1 to result

MOVE to next term in poly1

ELSE:

ADD term2 to result

MOVE to next term in poly2

STEP 3: Add remaining terms from both polynomials

4. Polynomial Multiplication Algorithm

Array Representation

STEP 1: Declare $\text{result}[2*\text{MAX_DEGREE}+1]$ initialized to 0

STEP 2: For $i = 0$ to deg1 :

For $j = 0$ to deg2 :

$\text{result}[i+j] += \text{poly1}[i] * \text{poly2}[j]$

Structure Representation

STEP 1: Initialize empty result list

STEP 2: For each term in poly1:

For each term in poly2:

```
new_coeff = coeff1 * coeff2
```

```
new_exp = exp1 + exp2
```

ADD/UPDATE result with new term

STEP 3: Combine like terms in result

STEP 4: Sort result by exponent

5. Main Program Flow

STEP 1: Choose representation type (Array/Structure)

STEP 2: Input polynomial(s)

STEP 3: Select operation (Addition/Multiplication)

STEP 4: Perform chosen operation

STEP 5: Display input and result polynomials

STEP 6: Repeat or exit

➤ CODE –

1. polyArray.cpp

```
#include<iostream>
using namespace std;

int main(){
    const int MAXIMUM=100;
    int poly[MAXIMUM+1]={0};
    int poly2[MAXIMUM+1]={0};

    int degree;
    cout<<"Enter the degree of first polynomial: "<<endl;
    cin>>degree;
    cout<<"Enter the coefficients of first polynomial:"<<endl;
    for(int i=degree;i>=0;i--){
        cout<<"Coefficient of x^"<<i<<": ";
        cin>>poly[i];
    }
    cout<<"The first polynomial is: "<<endl;
    for(int i=degree;i>=0;i--){
        if(poly[i]!=0){
            cout<<poly[i];
            if(i>0){
                cout<<"x^"<<i;
            }
            if(i!=0){
                cout<<"+";
            }
        }
    }

    cout<<endl;
```

```

cout<<"Enter degree of second polynomial: "<<endl;
int degree2;
cin>>degree2;
cout<<"Enter the coefficients of second polynomial:"<<endl;
for(int i=degree2;i>=0;i--){
    cout<<"Coefficient of x^"<<i<<": ";
    cin>>poly2[i];
}
cout<<"The second polynomial is: "<<endl;
for(int i=degree2;i>=0;i--){
    if(poly2[i]!=0){
        cout<<poly2[i];
        if(i>0){
            cout<<"x^"<<i;
        }
        if(i!=0){
            cout<<"+";
        }
    }
}

cout<<endl;

int result[MAXIMUM+1]={0};
int m=max(degree, degree2);
for(int i=0;i<=m;i++){
    result[i]=poly[i]+poly2[i];
}

cout<<"The sum of polynomial is: "<<endl;
for(int i = m;i>=0;i--){
    if(result[i]!=0){
        cout<<result[i];
        if(i>0){
            cout<<"x^"<<i;
        }
        if(i!=0){
            cout<<"+";
        }
    }
}

for(int i=0;i<=m;i++){
    for(int j=0;j<=m;j++){
        if(i+j<=m){
            result[i+j]+=poly[i]*poly2[j];
        }
    }
}

cout<<"The product of polynomial is: "<<endl;
for(int i = m;i>=0;i--){
    if(result[i]!=0){
        cout<<result[i];
        if(i>0){
            cout<<"x^"<<i;
        }
        if(i!=0){
            cout<<"+";
        }
    }
}

```

```

    }
  }
}

```

2. PolyStruct.cpp

```

#include <iostream>
using namespace std;

struct Term {
    int coeff;
    int exp;
};

int main() {
    Term poly1[] = {{3,2}, {2,1}, {1,0}};
    Term poly2[] = {{2,1}, {3,0}};
    int n1 = 3, n2 = 2;
    cout << "First polynomial: ";
    for(int i = 0; i < n1; i++) {
        cout << poly1[i].coeff << "x^" << poly1[i].exp;
        if(i != n1-1) cout << " + ";
    }
    cout << "\nSecond polynomial: ";
    for(int i = 0; i < n2; i++) {
        cout << poly2[i].coeff << "x^" << poly2[i].exp;
        if(i != n2-1) cout << " + ";
    }

    Term result[10];
    int n3 = 0;

    for(int i = 0; i < n1; i++) {
        result[n3++] = poly1[i];
    }
    for(int i = 0; i < n2; i++) {
        bool found = false;
        for(int j = 0; j < n3; j++) {
            if(poly2[i].exp == result[j].exp) {
                result[j].coeff += poly2[i].coeff;
                found = true;
                break;
            }
        }
        if(!found) {
            result[n3++] = poly2[i];
        }
    }
    cout << "\n\nSum: ";
    for(int i = 0; i < n3; i++) {
        cout << result[i].coeff << "x^" << result[i].exp;
        if(i != n3-1) cout << " + ";
    }
}

```

```

Term product[10];
int n4 = 0;

for(int i = 0; i < n1; i++) {
    for(int j = 0; j < n2; j++) {
        int new_coeff = poly1[i].coeff * poly2[j].coeff;
        int new_exp = poly1[i].exp + poly2[j].exp;
        bool found = false;
        for(int k = 0; k < n4; k++) {
            if(product[k].exp == new_exp) {
                product[k].coeff += new_coeff;
                found = true;
                break;
            }
        }
        if(!found) {
            product[n4].coeff = new_coeff;
            product[n4].exp = new_exp;
            n4++;
        }
    }
}
cout << "\nProduct: ";
for(int i = 0; i < n4; i++) {
    cout << product[i].coeff << "x^" << product[i].exp;
    if(i != n4-1) cout << " + ";
}
return 0;
}

```

➤ OUTPUT –

Array Representation

```

Enter the degree of first polynomial:
3
Enter the coefficients of first polynomial:
Coefficient of x^3: 6
Coefficient of x^2: 3
Coefficient of x^1: 9
Coefficient of x^0: 1
The first polynomial is:
6x^3+3x^2+9x^1+1
Enter degree of second polynomial:
3
Enter the coefficients of second polynomial:
Coefficient of x^3: 2
Coefficient of x^2: 5
Coefficient of x^1: 3
Coefficient of x^0: 7
The second polynomial is:
2x^3+5x^2+3x^1+7
The sum of polynomial is:
8x^3+8x^2+12x^1+8The product of polynomial is:
106x^3+61x^2+78x^1+15

```

Structure Representation

First polynomial: $3x^2 + 2x^1 + 1x^0$

Second polynomial: $2x^1 + 3x^0$

Sum: $3x^2 + 4x^1 + 4x^0$

Product: $6x^3 + 13x^2 + 8x^1 + 3x^0$

➤ CONCLUSION –

This practical lab demonstrates polynomial operations using array and structure representations, highlighting the trade-off between implementation simplicity and memory efficiency. The array approach excels for dense polynomials with direct index access, while the structure method better handles sparse polynomials through flexible term storage.